

Code review — teorie-incapsulare

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-09-30 — sha c588113

Fișa de referință: `exercitii-clase-obiecte.pdf` (20 de exerciții, clase și obiecte). Ce ai livrat în commit-ul „tema”: `Elev.cs`, `imprumut.cs`, `Autor.cs`, `Class2.cs` și 574 de linii noi în `App.cs`.

Poarta de compilare:  `dotnet build` — 0 erori, 13 warning-uri. Proiectul tău țintește `net10.0`, iar eu am SDK 8. Am copiat sursele într-un folder de lucru și le-am compilat pe `net8.0`; nicio linie din codul tău nu a fost schimbată.

Verificat prin rulare, nu prin citire. Am pornit programul de trei ori: așa cum e comis, apoi cu apelurile din `Main` repuse, apoi cu lista de cărți golită — exact verificarea pe care o cere checklist-ul fișei tale. Fiecare  de mai jos are dedesubt ce a tipărit programul.

Unde ai ajuns pe fișă: 13 exerciții din 20. Făcute: 1, 2, 3, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15. Pe jumătate: 4 și 5. Nefăcute: 16, 17, 19, 20. Parțial: 18.

Critice

B1 — Programul nu face nimic. Zero linii afișate.

`App.cs:12-23`

CUM E ACUM: am rulat exact ce ai comis. Rezultat:

```
$ dotnet run
$ echo "octeti afisati: $(dotnet run | wc -c)"
octeti afisati: 0
```

`Main` are toate cele nouă apeluri comentate. Cele 669 de linii de cod scrise de tine există în fișier, compilează, și nu se execută niciuna.

DE CE: `Main` nu e „locul unde stă programul”, e singura ușă prin care intră execuția. Runtime-ul .NET caută metoda `Main`, o apelează, și când ea se termină procesul se închide. O metodă pe care nu o cheamă nimeni, direct sau indirect din `Main`, e cod care nu va rula niciodată — compilatorul o traduce, o pune în executabil, și nimeni nu o deschide.

De-aia n-ai văzut niciun bug din cele de mai jos: nu că nu existau, ci că n-au avut ocazia să se manifeste. Programul a „mers” de fiecare dată, fiindcă un program care nu face nimic nu poate greși nimic.

Și este a doua oară. În `recapitulare.cpp` la C++ ai `main()` cu 90 de linii de teste comentate și una activă. Acolo rula ultimul lucru; aici nu mai rulează nici acela. Obiceiul e același: în loc să ștergi un test terminat, îl comentezi. După zece exerciții, fișierul e un cimitir și nu mai știi ce e viu.

FIX: `Main` cheamă, în ordine, exercițiile pe care vrei să le vezi. Când unul e gata și verificat, îl **ștergi** din `Main` — nu îl comentezi. Codul lui rămâne în metoda lui, iar `git log` îți ține minte că l-ai făcut.

```
static void Main(string[] args)
{
    carte();
    elev();
    imprumut();
}
```

Cu cele trei apeluri repuse, programul tipărește 407 linii. Toate -urile de jos ies la iveală de la prima rulare.

B2 — `ieftinire` nu ieftinește nimic. Exercițiul 4 nu e făcut.

`Carte.cs:48–55`

CUM E ACUM: o carte de 45 lei, după `ieftinire(24)` :

```
pret initial           : 45
reducere(20) intoarce  : 36  <- corect
pret dupa reducere    : 45  <- corect, ex 3 nu are voie sa schimbe
pret dupa ieftinire(24) : 45  <- ar fi trebuit 21
```

Pe ecran scrie „Inainte 45 / Dupa 21”, deci pare că a funcționat. Cartea rămâne 45.

DE CE: `int pret2 = pret - suma;` creează o **variabilă nouă**, locală metodei. `pret2` trăiește între acolada de la `:49` și cea de la `:54`; când metoda se termină, dispare. Câmpul `pret` al obiectului n-a fost atins niciodată — tu ai calculat cât *ar fi* prețul, l-ai tipărit, și l-ai aruncat.

Asta e exact diferența pe care o cere exercițiul 4 față de 3, și pe care ai scris-o singură în comentariul de la `:55`: „Ex 3 calculează și face return, fără să modifice prețul, iar Ex 4 modifică prețul”. Ai înțeles regula corect. Doar că amândouă metodele fac acum același lucru — niciuna nu modifică nimic.

`ieftinire` diferă de `reducere` doar prin faptul că tipărește în loc să întoarcă.

Ca să *modifici* obiectul, atribuirea trebuie să cadă pe **câmp**, nu pe o variabilă locală: `pret = ...`. Un câmp aparține obiectului și supraviețuiește apelului; o variabilă locală aparține apelului și moare cu el.

Și un al doilea lucru, pe care fișa îl cere peste tot: `Console.WriteLine` nu are ce căuta în `Carte`. Metoda modifică și întoarce noul preț; cine vrea să vadă „înainte” și „după” le tipărește din `App`, unde are amândouă valorile.

FIX:

```
public int ieftinire(int suma)
{
    pret = pret - suma;
    return pret;
}
```

Iar în `App`, unde ai și valoarea veche:

```
Console.WriteLine("Inainte " + carti[i].pret);
Console.WriteLine("Dupa " + carti[i].ieftinire(24));
```

Verificat: după fix, `pret dupa ieftinire(24) : 21`.

B3 — Prețul mediu iese 38 în loc de 38.75, și schimbă răspunsul la exercițiul 10.

App.cs:545

CUM E ACUM: cele opt cărți costă 45, 35, 25, 40, 38, 42, 30, 55. Suma e 310, media reală 38.75.

Programul tipărește:

```
Pretul mediu 38
Poezii
Amintiri din copilarie
Micul Print
Cate sub medie 3
```

După fix:

```
Pretul mediu 38.75
Poezii
Amintiri din copilarie
Maitreyi
Micul Print
Cate sub medie 4
```

Nu e doar un număr afișat urât. „**Maitreyi**” costă 38 și e sub medie, dar programul n-o numără — fiindcă a comparat-o cu 38, nu cu 38.75. Răspunsul la exercițiul 10 e greșit cu o carte.

DE CE: `double medie = suma / carti2.Count;` — `suma` e `int`, `Count` e `int`. C# alege operația în funcție de **tipurile operanzilor**, nu de tipul în care pui rezultatul. Două `int` înseamnă împărțire întreagă: $310 / 8 = 38$, restul se aruncă pe loc. Abia apoi 38 e convertit în `double` și pus în `medie`. `double` de la stânga egalului ajunge prea târziu — paguba s-a produs deja la dreapta.

Și știi asta: în `Elev.cs:25` ai scris `return (nota1 + nota2 + nota3) / 3.00;` — **exact trucul corect**, `3.00` în loc de `3`, ca împărțirea să fie între un `int` și un `double` și rezultatul să fie `double`. Acolo ai făcut-o bine. Aici, la a doua împărțire, ai uitat-o.

Același lucru e și în `Carte.cs:45`: `pret - pret * procent / 100` pe o carte de 38 lei cu 20% dă 31, nu 30.40 — $38 * 20 = 760$, $760 / 100 = 7$ (nu 7.6), $38 - 7 = 31$.

FIX: forțează un operand la `double`, în amândouă locurile.

```
double medie = (double)suma / carti2.Count;
```

```
public double reducere(int procent)
{
    return pret - pret * procent / 100.0;
}
```

Verificat: 38.75 și 4 cărți sub medie; `reducere(20)` pe 38 lei dă 30.4.

B4 — Primul elev nu se afișează niciodată. Și e chiar șefa clasei.

App.cs:696

CUM E ACUM: lista are șase elevi. Programul afișează cinci:

```
Nume Ionescu Vlad
Nume Marinescu Elena
Nume Dumitrescu Andrei
Nume Vasilescu Maria
Nume Radu Mihai

Media clasei 6.28
Cea mai mare medie Popescu Ana 9
```

„Popescu Ana” nu apare în listă, dar apare ca elevul cu media cea mai mare. E în listă, e în medie, e în numărătoarea promoțiilor — doar afișarea o sare.

DE CE: `for (int i = 1; i < 6; i++)`. Într-un `List<T>`, primul element e la poziția **0**, nu 1. Pornind de la `i = 1` sari peste `elevi2[0]`, iar oprindu-te la `i < 6` te oprești la ultimul — deci parcurgi cinci din șase, decalat cu unu.

Că bug-ul e doar la afișare și nu la socoteli e o coincidență: celelalte trei bucle, de la `:703`, `:712` și `:724`, pornesc corect de la 0. De-aia media clasei e bună și Ana e găsită ca maximă. Un singur `1` scris în loc de `0`, într-un singur loc.

Și e a doua oară în două limbaje: la C++, în runda 1, `B6` era exact asta — baleiere peste `ct[10]` pornită de la `i = 1`, care făcea cifra 0 invizibilă.

Al doilea lucru, din aceeași familie: **6 scris de mână în patru bucle**. Dacă mâine adaugi un al șaptelea elev, lista are 7 și programul citește 6 — iar bug-ul nu-ți spune nimic, doar dă un răspuns greșit. Lungimea listei o știe lista: `elevi2.Count`. Același lucru la `:806`, unde ai `i < 5` pentru cele cinci împrumuturi.

FIX:

```
for (int i = 0; i < elevi2.Count; i++)
```

și la fel în celelalte trei bucle din `elev()` plus în `imprumut()`, iar `double mediaClasei = suma / elevi2.Count;` în loc de `/ 6`.

Verificat: după fix se afișează toți șase, media rămâne 6.28.

B5 — Pe listă goală programul crapă. Garda există, dar la exercițiul greșit.

`App.cs:508` și `App.cs:545`

CUM E ACUM: checklist-ul de la finalul fișei cere „ce face programul pe o listă goală”. Am golit lista de cărți și am rulat:

```
-----TOATE CARTILE-----
-----DISPONIBILE-----
Lista e goala.
Unhandled exception. System.DivideByZeroException:
  Attempted to divide by zero.
  at teorie_incapsulare.App.carte() in App.cs:line 545
```

Programul scrie frumos „Lista e goala.”, și opt linii mai jos moare.

DE CE: la exercițiul 9 ai pus garda — `if (carti2.Count == 0)` la `:508` — și e scrisă corect. Dar ea acoperă numai blocul lui `else`, care se încheie la `:537`. Exercițiul 10 începe la `:538`, în afara gării, și ajunge la `suma / carti2.Count`. Cu `Count == 0` asta e împărțire de întregi la zero, iar în C# împărțirea întreagă la zero nu dă infinit, ci aruncă `DivideByZeroException` și oprește procesul.

(Dacă ar fi fost `double` împărțit la `double`, ai fi văzut `NaN` și programul ar fi mers înainte. Deci B3 și B5 sunt același `int` care face două rele diferite: acolo trunchiază, aici aruncă.)

Ăsta e tiparul pe care vreau să-l vezi, mai mult decât bug-ul: **ai știut regula și ai aplicat-o într-un singur loc**. La fel ca la B3, unde `/3.00` e corect în `Elev` și lipsește în `App`. Nu-ți lipsește cunoștința, îți lipsește al doilea pas — după ce repari un loc, cauți celelalte locuri unde e aceeași problemă.

FIX: o singură gardă, la începutul metodei, înainte de orice socoteală:

```
if (carti2.Count == 0)
{
    Console.WriteLine("Lista e goala.");
    return;
}
```

și scoți `if`-ul de la `:508`, care devine de prisos.

Verificat: cu lista golită, programul scrie „Lista e goala.” și se termină normal.

🟡 Importante

M1 — Exercițiul 5 e pe jumătate. `Carte.cs:58` — `veche()` e corectă (peste 50 de ani). „Ocazie bună” (disponibilă **și** sub 50 lei) nu există. Metodele publice din `Carte` sunt: `DescriereCarte`, `AnideDeLaAparitie`, `reducere`, `ieftinire`, `veche`, `Descrierecarte`. Fișa cerea **două** metode `bool`, ai scris una. E a patra rundă la rând în care un exercițiu cu două cerințe se încheie pe prima.

M2 — Exercițiul 16 e abandonat prin comentare. `Autor.cs` — tot fișierul, 23 de linii, e comentariu; zero linii de cod. Plus `App.cs:755–766`, metoda `Autor()`, la fel. Am comentat-o și am compilat-o separat: **merge, 0 erori**. Deci nu te-a oprit compilatorul. Ce te-a oprit e că exercițiul cere să înlocuiești câmpul `public string autor` cu un `Autor` — și în clipa aceea cele 19 linii `ca1.autor = "..."` din `App.cs` nu mai compilează. Asta nu e un semn că ai greșit: e primul exercițiu în care o schimbare într-o clasă se propagă la toți cei care o folosesc. Costul e real (trei linii pe carte în loc de una) și e exact motivul pentru care există constructori — lucrul care urmează la lecția următoare.

M3 — Exercițiile 17, 19 și 20 nu sunt începute. `Biblioteca` cu lista ca câmp al ei, mutarea cărții între raft și împrumutate, meniul cu `switch`. 17 e cel care contează: fișa spune că după ce muți

metodele în `Biblioteca` , „în `App` nu mai rămâne nicio buclă care umblă prin listă”. Acum în `App` sunt 14 bucle.

M4 — Exercițiul 18 e pe jumătate. `imprumut.cs:9–11` ține `string nume` și `string carte` . Fișa cere un obiect `Elev` și un obiect `Carte` . Socoteala e corectă (`intarziere` și `penalizare` dau răspunsurile bune pe toate cele cinci împrumuturi), dar clasa nu face ce trebuia să exerseze: clase care se cunosc între ele. Aceeași cauză ca M2 — de fiecare dată când un exercițiu cere un obiect ca câmp, apare un `string` .

M5 — `Descrierecarte()` e `DescriereCarte()` minus două linii. `Carte.cs:70–82` vs `Carte.cs:21–34` . Aceleași cinci linii copiate, doar fără „Ani de la aparitie” și „Pret dupa reducere”. Dacă mâine adaugi un câmp la `Carte` , îl scrii în două locuri sau îl uiți într-unul. Cea scurtă poate fi cea de bază, iar cea lungă o cheamă: `return Descrierecarte() + "Ani de la aparitie ..." + ...` .

M6 — `Carte()` și `carte()` sunt aceeași metodă de două ori. `App.cs:206–315` și `App.cs:415–585` . 47 de linii identice caracter cu caracter — aceleași cărți, aceleași prețuri, o singură diferență de diacritice la `ca1.autor` . La fel `Elev()` (`:317`) și `elev()` (`:644`): aceiași șase elevi, cu variabilele redenumite din `e1` în `e11` . Fișa spunea în prima casetă: „nu porni de la zero la fiecare exercițiu”. O singură metodă care construiește raftul și îl **întoarce**, iar exercițiile 8–13 și 15 o cheamă.

M7 — `DescriereCarte()` calculează reducerea în interiorul descrierii. `Carte.cs:29–30` . Orice afișare a unei cărți tipărește și „Ani de la aparitie” și „Pret dupa reducere 36” — inclusiv la exercițiul 11 (caută după titlu), 12 (cărțile autorului) și 13 (sortare), unde reducerea n-are nicio treabă. O descriere descrie; socotelile se cer separat, când le vrei.

M8 — Exercițiul 11 întoarce text, nu carte. `App.cs:587` — `CautaDupaTitlu` întoarce `string` . Fișa cere să întoarcă cartea, iar cel care a cerut-o să afle că nu există. Cu `string` nu poți face nimic cu cartea găsită: n-o poți ieftini, n-o poți muta în altă listă. Exercițiul 19 ar fi imposibil așa.

M9 — `promovalabilitate()` întoarce `string` . `Elev.cs:28` — „promovat/a” / „nepromovat/a”. Fișa cere răspunsul la o întrebare, deci `bool` ; textul se compune în `App` . Dovada că deranjează: la `:726` în `elev()` ai fost nevoită să rescrii `if (elevi2[i].media() >= 5)` în loc să chemi metoda, fiindcă nu puteai pune un `string` într-un `if` . Și media se afișează `4.333333333333333` , nu `4.33` — `Math.Round(x, 2)` , pe care îl folosești corect la `:709` .

M10 — 236 de linii de atribuire câmp cu câmp, pentru 55 de obiecte. 35% din `App.cs` . Restanță din 22 septembrie, atunci erau 60 de linii. Cauza nu e lenea, e că nicio clasă n-are constructor — de-aia și cele 13 warning-uri CS8618. O singură cauză, nu 55 de simptome; se rezolvă la lecția următoare, nu acum.

Cleanups

C1 `Class1.cs` și `Class2.cs` — două clase complet goale, comise. De șters. **C2** `imprumut` cu literă mică → `warning CS8981: The type name 'imprumut' only contains lower-cased ascii characters. Such names may become reserved for the language.` Clasele se scriu cu majusculă: `Imprumut`. **C3** `Ttitlu` în loc de `Titlu`, în două locuri: `Carte.cs:24` și `Carte.cs:74` — copiat o dată, greșit de două ori. **C4** 13 warning-uri `CS8618` la build. Dispar cu constructori (vezi M10). **C5** `using` nefolosite: `System.Security.Cryptography.X509Certificates`, `System.Runtime.InteropServices`, `static System.Net.Mime.MediaTypeNames` — `App.cs:3-6` și `Carte.cs:3-4`. Le pune Visual Studio la autocompletare; se șterg. Iar `System`, `System.Collections.Generic` și `System.Text` sunt deja incluse prin `ImplicitUsings` din `.csproj`. **C6** 35 de linii comentate în `App.cs` și 24 de linii albe după acolada finală (`:817-839`), în afara namespace-ului. **C7** `User.cs` și `Car.cs` rămân din tema precedentă și nu mai sunt apelate. `Car` n-a fost niciodată pe fișă. Nu deranjează, dar `User()` (124 de linii) și `Car()` (50) sunt cod pe care nu-l mai rulezi.

Ce e bine — și e mai mult decât pare

Exercițiul 15 e cel mai bun lucru din tema asta. `App.cs:737-753`. Nu doar codul, ci răspunsul scris: „ambele liste țin referință spre același obiect din heap. În C++ cu obiect pe stivă s-ar fi făcut o copie în fiecare listă”. Țasta e răspunsul complet, inclusiv partea de C++. Era singurul exercițiu care cerea explicație în cuvinte și e singurul la care n-am ce să adaug.

Sortarea de la exercițiul 13 e corectă. `App.cs:627-641`. Două bucle, schimb prin `aux`, fără LINQ și fără `Sort` — exact cum cerea fișa. Am rulat-o, ordonează bine.

`Elev.media()` folosește `/3.00`. `Elev.cs:25`. Nu e o întâmplare — e trucul pe care mulți îl ratează prima dată. Îl ai; problema de la B3 e că nu l-ai dus la a doua împărțire.

`notaMax()` și `notaMin()` sunt scrise ca funcții care întorc, nu care afișează. `Elev.cs:40-65`. Fișa cerea la exercițiul 7 „fără să repeți codul de trei ori în App” — și n-ai repetat.

Garda de la exercițiul 9. `App.cs:508`. Te-ai gândit singură la lista goală, ceea ce nu ți-a cerut nimeni explicit în enunț. B5 nu e că n-ai știut — e că ai oprit-o prea repede.

Și ai livrat mult. 13 exerciții din 20, 574 de linii într-un commit, pe un limbaj pe care îl înveți de o săptămână. Pe majoritatea lucrurilor cerute, socoteala e corectă.

Before / After

B1 — Main gol

Acum — App.cs:12	Cum ar trebui
<pre>// Useri(); //Car(); //Carte(); //Elev(); //carte(); // elev(); //carteEx15(); // Autor(); //imprumut(); — nouă apeluri comentate, 0 octeți afișați</pre>	<pre>carte(); elev(); imprumut(); — apeluri active; ce e terminat se șterge, nu se comentează</pre>

B2 — ieftinire nu modifică prețul

Acum — Carte.cs:48	Cum ar trebui
<pre>public void ieftinire(int suma) { Console.WriteLine("Inainte " + pret); int pret2 = pret - suma; Console.WriteLine("Dupa " + pret2); }</pre>	<pre>public int ieftinire(int suma) { pret = pret - suma; return pret; } — atribuirea cade pe câmp, afișarea pleacă în App</pre>

B3 — împărțire între întregi

Acum	Cum ar trebui
<pre>App.cs:545 — double medie = suma / carti2.Count; → 38</pre>	<pre>double medie = (double)suma / carti2.Count; → 38.75</pre>
<pre>Carte.cs:45 — return pret - pret * procent / 100; → 31</pre>	<pre>return pret - pret * procent / 100.0; → 30.4 (și tipul metodei devine double)</pre>

B4 — bucla pornită de la 1

Acum — App.cs:696	Cum ar trebui
<pre>for (int i = 1; i < 6; i++) — sare primul elev, 6 scris de mână</pre>	<pre>for (int i = 0; i < elevi2.Count; i++) — și la fel în :703 , :712 , :724 , :806</pre>

B5 — garda pe listă goală

Acum — App.cs:508	Cum ar trebui
<pre>if (carti2.Count == 0) { ... } else { /* ex 9 */ } — apoi ex 10 socotește în afara gărzii și crapă</pre>	o gardă la începutul metodei, cu <code>return</code> , care acoperă tot ce urmează

Întrebări

- În `Carte.cs`, `reducere` și `ieftinire` fac amândouă `pret - ceva`. Una lasă cartea neschimbată, cealaltă ar trebui să o schimbe. Uită-te la ce scrie în stânga semnului `=` în fiecare: ce e `pret2`, și cât trăiește? Dacă ai muta `int pret2 = pret - suma;` în `reducere`, s-ar schimba ceva la ce întoarce ea?
- `Elev.cs:25` are `/ 3.00`, iar `App.cs:545` are `/ carti2.Count`. Ambele pun rezultatul în ceva de tip `double`, dar numai una dă răspunsul corect. Ce se întâmplă *întâi*: împărțirea, sau punerea rezultatului în `double`? Și de ce contează ordinea?
- Ai scris garda pentru lista goală la exercițiul 9, dar programul crapă la exercițiul 10. Fără să te uiți la cod: mai sunt locuri în tema asta unde ai știut ceva și l-ai aplicat o singură dată? (Uită-te la `/ 3.00` și la `i = 0`.) Ce ai putea face, după ce repari un loc, ca să găsești celelalte?

